

DSC 540 - Data Preparation

Weeks 9 & 10- Term Project: Milestone 4

Manoj K Kola

Connecting to an API/Pulling in the Data and Cleaning/Formatting

Dataset: OpenWeatherMap Current Weather API (10 cities)

In this section, we prepare our environment to work with weather data from the OpenWeatherMap API.

- **Requests Library:** This library allows us to send HTTP requests to APIs and receive responses.
- **Pandas:** A powerful library for data handling and manipulation, ideal for working with tabular data.
- **NumPy:** Provides support for numerical operations and efficient mathematical computations.
- **Datetime:** Helps in working with dates and times, which is essential for time-based data analysis.
- **API Key:** An access key from OpenWeatherMap required to authenticate and retrieve weather data.

```
In [105... # Import Library to make API requests
import requests

# Import pandas for data handling
import pandas as pd

# Import numpy for numerical operations
import numpy as np

# Import datetime to work with date and time
from datetime import datetime

# Store your OpenWeatherMap API key
API_KEY = "78f4dfcd2917867d3c49d4a49fe4990f"
```

Step 0: Define 10 Cities with City, State, Country

In this section, we define a list of cities along with their corresponding states and country codes. This list will help retrieve weather data for multiple locations.

- **City:** The name of the city we want to access data for.
- **State:** The two-letter state abbreviation (for US cities) to ensure accurate location mapping.
- **Country:** The two-letter country code, following the ISO 3166 standard, to specify the country.

This structured list makes it easy to iterate over multiple locations when pulling data from the weather API.

```
In [106... # Create a list of cities with their state and country
locations = [
    # Ohio
    {"city": "Columbus", "state": "OH", "country": "US"},

    # New York
    {"city": "New York", "state": "NY", "country": "US"},

    # Illinois
    {"city": "Chicago", "state": "IL", "country": "US"},

    # California
    {"city": "Los Angeles", "state": "CA", "country": "US"},

    # Texas
    {"city": "Houston", "state": "TX", "country": "US"},

    # Arizona
    {"city": "Phoenix", "state": "AZ", "country": "US"},

    # Pennsylvania
    {"city": "Philadelphia", "state": "PA", "country": "US"},

    # Texas
    {"city": "San Antonio", "state": "TX", "country": "US"},

    # California
    {"city": "San Diego", "state": "CA", "country": "US"},

    # Texas
    {"city": "Dallas", "state": "TX", "country": "US"}
]
```

Step 1: API Calls for All Cities

This section demonstrates how to collect current weather data for multiple locations using the OpenWeatherMap API.

- **Records List:** An empty list is created to store weather data retrieved from the API for each location.
- **Loop Through Locations:** We iterate over a list of locations, extracting the city, state, and country for each entry.

- **Build API URL:** A URL is constructed for each location, specifying the city, state, country, API key, and units (Fahrenheit in this case).
- **API Request:** The `requests` library sends a request to the OpenWeatherMap API and retrieves the response.
- **Convert to JSON:** The API response is converted into JSON format for easier handling.
- **Add Input Metadata:** Original location information (city, state, country) is added to each JSON record to keep track of the source.
- **Append to Records:** Each JSON record is added to the `records` list.
- **Create DataFrame:** The list of JSON records is normalized and converted into a Pandas DataFrame for easy data manipulation and analysis.
- **Display Data:** The DataFrame is displayed to inspect the collected weather data.

```
In [110... # Create an empty list to store API results
records = []

# Loop through each location in the list
for loc in locations:

    # Get city, state, and country from dictionary
    city = loc["city"]
    state = loc["state"]
    country = loc["country"]

    # Build the API URL for current weather (units in Fahrenheit)
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city},{state},{country}&units=fahrenheit"

    # Send request to the API
    response = requests.get(url)

    # Convert response to JSON format
    data = response.json()

    # Add original input values to the response
    data["input_city"] = city
    data["input_state"] = state
    data["input_country"] = country

    # Add the data to the records list
    records.append(data)

# Convert list of JSON records into a DataFrame
df = pd.json_normalize(records)

# Display the DataFrame
df
```

Out[110...

	weather	base	visibility	dt	timezone	id	name	cod	inp
0	{'id': 802, 'main': 'Clouds', 'description': ...}	stations	10000	1771639971	-18000	4509177	Columbus	200	Col
1	{'id': 701, 'main': 'Mist', 'description': 'm...	stations	10000	1771639856	-18000	5128581	New York	200	Ne
2	{'id': 804, 'main': 'Clouds', 'description': ...}	stations	10000	1771639831	-21600	4887398	Chicago	200	C
3	{'id': 800, 'main': 'Clear', 'description': '...	stations	10000	1771639722	-28800	5368361	Los Angeles	200	Los A
4	{'id': 804, 'main': 'Clouds', 'description': ...}	stations	10000	1771639739	-21600	4699066	Houston	200	H
5	{'id': 802, 'main': 'Clouds', 'description': ...}	stations	10000	1771639974	-25200	5308655	Phoenix	200	P
6	{'id': 701, 'main': 'Mist', 'description': 'm...	stations	2012	1771639777	-18000	4560349	Philadelphia	200	Philac
7	{'id': 803, 'main': 'Clouds', 'description': ...}	stations	10000	1771639850	-21600	4726206	San Antonio	200	A
8	{'id': 801, 'main': 'Clouds', 'description': ...}	stations	10000	1771639871	-28800	5391811	San Diego	200	San
9	{'id': 804, 'main': ...}	stations	10000	1771639977	-21600	4684904	Dallas	200	

weather	base	visibility	dt	timezone	id	name	cod	inp
'Clouds', 'description': ...								

Step 2: Expand Weather Column (Fix Missing Data)

In this step, we transform the nested weather information in our dataset into a more usable format.

- **Extract Weather Data:** The `weather` column contains a list of weather objects. We extract the first object from each list and expand it into separate columns.
- **Rename Columns:** To make the column names more readable, we rename them as:
 - `id` → `Weather_Id`
 - `main` → `Weather_Main`
 - `description` → `Weather_Description`
 - `icon` → `Weather_Icon`
- **Update DataFrame:** We remove the original `weather` column and join the newly expanded columns to the main dataframe.
- **Display Data:** Finally, we display the updated dataframe to verify the changes.

In [111...

```
# Extract the first weather object from the list and convert it into separate column
weather_expanded = df["weather"].apply(lambda x: x[0]).apply(pd.Series)

# Rename the weather columns to make them more readable
weather_expanded = weather_expanded.rename(columns={
    "id": "Weather_Id",
    "main": "Weather_Main",
    "description": "Weather_Description",
    "icon": "Weather_Icon"
})

# Remove the original "weather" column and join the new weather columns to the prim
df = pd.concat([df.drop(columns=["weather"]), weather_expanded], axis=1)

# Display updated dataframe
df
```

Out[111...

	base	visibility	dt	timezone	id	name	cod	input_city	input
0	stations	10000	1771639971	-18000	4509177	Columbus	200	Columbus	
1	stations	10000	1771639856	-18000	5128581	New York	200	New York	
2	stations	10000	1771639831	-21600	4887398	Chicago	200	Chicago	
3	stations	10000	1771639722	-28800	5368361	Los Angeles	200	Los Angeles	
4	stations	10000	1771639739	-21600	4699066	Houston	200	Houston	
5	stations	10000	1771639974	-25200	5308655	Phoenix	200	Phoenix	
6	stations	2012	1771639777	-18000	4560349	Philadelphia	200	Philadelphia	
7	stations	10000	1771639850	-21600	4726206	San Antonio	200	San Antonio	
8	stations	10000	1771639871	-28800	5391811	San Diego	200	San Diego	
9	stations	10000	1771639977	-21600	4684904	Dallas	200	Dallas	



In [112...

```
# Show all column names in the dataframe  
list(df.columns)
```

```
Out[112... ['base',
            'visibility',
            'dt',
            'timezone',
            'id',
            'name',
            'cod',
            'input_city',
            'input_state',
            'input_country',
            'coord.lon',
            'coord.lat',
            'main.temp',
            'main.feels_like',
            'main.temp_min',
            'main.temp_max',
            'main.pressure',
            'main.humidity',
            'main.sea_level',
            'main.grnd_level',
            'wind.speed',
            'wind.deg',
            'clouds.all',
            'sys.type',
            'sys.id',
            'sys.country',
            'sys.sunrise',
            'sys.sunset',
            'wind.gust',
            'Weather_Id',
            'Weather_Main',
            'Weather_Description',
            'Weather_Icon']
```

Step 3: Cleaning and Renaming DataFrame Columns

In this step, we improve the readability and consistency of our dataset's column names.

- **Rename Columns:** We assign more meaningful names to each column, e.g., `input_city` becomes `City` and `main.temp` becomes `Temperature_F`.
- **Format Column Names:** We standardize column names by replacing underscores with spaces, capitalizing each word, and then converting spaces back to underscores. This ensures consistency across all columns.
- **Confirmation Message:** A simple message confirms that the renaming process has been completed.
- **Display DataFrame:** The updated dataset is displayed so we can verify that the column names are now clear and readable.

```
In [113... # Rename columns to make them easier to read
df = df.rename(columns={
    "input_city": "City",
    "input_state": "State",
```

```

    "input_country": "Country",
    "sys.country": "Country_Code",
    "main.temp": "Temperature_F",
    "main.feels_like": "Feels_Like_F",
    "main.temp_min": "Temp_Min_F",
    "main.temp_max": "Temp_Max_F",
    "main.pressure": "Pressure",
    "main.humidity": "Humidity_Pct",
    "main.sea_level": "Sea_level",
    "main.grnd_level": "Grnd_level",
    "wind.speed": "Wind_Speed_Mph",
    "wind.deg": "Wind_Direction_Deg",
    "wind.gust": "Gust"
})

# Clean column names by formatting them consistently
df.columns = [col.replace("_", " ").title().replace(" ", "_") for col in df.columns]

# Print confirmation message
print("Renamed the columns\n")

# Display updated dataframe
df

```

Renamed the columns

Out[113...

	Base	Visibility	Dt	Timezone	Id	Name	Cod	City	State
0	stations	10000	1771639971	-18000	4509177	Columbus	200	Columbus	OH
1	stations	10000	1771639856	-18000	5128581	New York	200	New York	NY
2	stations	10000	1771639831	-21600	4887398	Chicago	200	Chicago	IL
3	stations	10000	1771639722	-28800	5368361	Los Angeles	200	Los Angeles	CA
4	stations	10000	1771639739	-21600	4699066	Houston	200	Houston	TX
5	stations	10000	1771639974	-25200	5308655	Phoenix	200	Phoenix	AZ
6	stations	2012	1771639777	-18000	4560349	Philadelphia	200	Philadelphia	PA
7	stations	10000	1771639850	-21600	4726206	San Antonio	200	San Antonio	TX
8	stations	10000	1771639871	-28800	5391811	San Diego	200	San Diego	CA
9	stations	10000	1771639977	-21600	4684904	Dallas	200	Dallas	TX



In [114...

```

# Show all column names in the dataframe, verify that the columns were renamed correctly
list(df.columns)

```

```
Out[114... ['Base',
            'Visibility',
            'Dt',
            'Timezone',
            'Id',
            'Name',
            'Cod',
            'City',
            'State',
            'Country',
            'Coord.Lon',
            'Coord.Lat',
            'Temperature_F',
            'Feels_Like_F',
            'Temp_Min_F',
            'Temp_Max_F',
            'Pressure',
            'Humidity_Pct',
            'Sea_Level',
            'Grnd_Level',
            'Wind_Speed_Mph',
            'Wind_Direction_Deg',
            'Clouds.All',
            'Sys.Type',
            'Sys.Id',
            'Country_Code',
            'Sys.Sunrise',
            'Sys.Sunset',
            'Gust',
            'Weather_Id',
            'Weather_Main',
            'Weather_Description',
            'Weather_Icon']
```

Step 4: Standardizing Text Fields in Weather Data

In this section, we clean and standardize the text fields in our weather dataset to ensure consistency and readability:

- **City:** Format the city names so that the first letter of each word is capitalized and any extra spaces are removed.
- **State:** Convert state codes to uppercase and remove extra spaces for uniformity.
- **Country:** Convert country codes to uppercase and remove extra spaces for consistency.
- **Weather Description:** Make weather descriptions more readable by applying title case formatting.
- **Confirmation Message:** A message is printed to confirm that the text fields have been standardized.
- **Display Cleaned Columns:** Show the cleaned `City`, `State`, `Country`, and `Weather_Description` columns for verification.

```
In [115... # Fix city name formatting (make first letter capital and remove extra spaces)
df["City"] = df["City"].str.title().str.strip()
```

```

# Convert state to uppercase and remove extra spaces
df["State"] = df["State"].str.upper().str.strip()

# Convert country to uppercase and remove extra spaces
df["Country"] = df["Country"].str.upper().str.strip()

# Make weather description more readable (Title Case)
df["Weather_Description"] = df["Weather_Description"].str.title()

# Print confirmation message
print("Standardized the text fields\n")

# Display cleaned columns
df[["City", "State", "Country", "Weather_Description"]]

```

Standardized the text fields

Out[115...

	City	State	Country	Weather_Description
0	Columbus	OH	US	Scattered Clouds
1	New York	NY	US	Mist
2	Chicago	IL	US	Overcast Clouds
3	Los Angeles	CA	US	Clear Sky
4	Houston	TX	US	Overcast Clouds
5	Phoenix	AZ	US	Scattered Clouds
6	Philadelphia	PA	US	Mist
7	San Antonio	TX	US	Broken Clouds
8	San Diego	CA	US	Few Clouds
9	Dallas	TX	US	Overcast Clouds

Step 5: Converting Unix Timestamps to Readable Date & Time

In this section, we convert Unix timestamps from the weather dataset into human-readable date and time formats.

- **Convert Timestamp:** The `Dt` column contains the main data timestamp in Unix format. We convert it to a standard date-time format.
- **Convert Sunrise & Sunset:** The `Sys.Sunrise` and `Sys.Sunset` columns contain sunrise and sunset times in Unix format. Converting these makes it easier to read and analyze.
- **Confirmation Message:** After conversion, a message is printed to confirm that the timestamps have been successfully converted.
- **Display Data:** Finally, we display selected columns, including city, state, country, timestamp, sunrise, and sunset, for verification.

```
In [98]: # Convert Unix timestamp to readable date and time
df["Timestamp"] = pd.to_datetime(df["Dt"], unit="s")

# Convert sunrise time to a readable format
df["Sunrise"] = pd.to_datetime(df["Sys.Sunrise"], unit="s")

# Convert sunset time to a readable format
df["Sunset"] = pd.to_datetime(df["Sys.Sunset"], unit="s")

# Print confirmation message
print("Converted the timestamps\n")

# Display selected columns
df[["City", "State", "Country", "Timestamp", "Sunrise", "Sunset"]]
```

Converted the timestamps

```
Out[98]:
```

	City	State	Country	Timestamp	Sunrise	Sunset
0	Columbus	OH	US	2026-02-21 01:35:59	2026-02-20 12:18:26	2026-02-20 23:13:20
1	New York	NY	US	2026-02-21 01:36:23	2026-02-20 11:43:27	2026-02-20 22:36:23
2	Chicago	IL	US	2026-02-21 01:36:10	2026-02-20 12:39:27	2026-02-20 23:29:31
3	Los Angeles	CA	US	2026-02-21 01:37:38	2026-02-20 14:32:27	2026-02-21 01:41:16
4	Houston	TX	US	2026-02-21 01:35:19	2026-02-20 12:56:36	2026-02-21 00:14:05
5	Phoenix	AZ	US	2026-02-21 01:39:13	2026-02-20 14:07:09	2026-02-21 01:17:13
6	Philadelphia	PA	US	2026-02-21 01:36:17	2026-02-20 11:47:07	2026-02-20 22:41:59
7	San Antonio	TX	US	2026-02-21 01:35:01	2026-02-20 13:08:47	2026-02-21 00:26:57
8	San Diego	CA	US	2026-02-21 01:35:54	2026-02-20 14:26:42	2026-02-21 01:38:19
9	Dallas	TX	US	2026-02-21 01:36:57	2026-02-20 13:05:19	2026-02-21 00:16:44

Step 6: Data Cleaning for Weather Dataset

In this section, we clean the weather dataset to ensure the values are valid and meaningful.

- **Handling Humidity Values:** Any humidity percentage values that are less than 0% or greater than 100% are invalid. These values are replaced with `NaN` to indicate missing

values.

- **Handling Wind Speed Values:** Wind speed cannot be negative, so any negative values are replaced with `NaN`.
- **Confirmation Message:** A simple message is printed to indicate that the invalid values have been cleaned successfully.
- **Verification:** Selected columns, including `City`, `State`, `Country`, `Humidity_Pct`, and `Wind_Speed_Mph`, are displayed to visually verify the cleaning process.

```
In [99]: import numpy as np # Make sure numpy is imported

# Replace invalid humidity values (<0 or >100) with NaN
df["Humidity_Pct"] = df["Humidity_Pct"].apply(
    lambda x: np.nan if (x < 0 or x > 100) else x
)

# Replace invalid wind speed values (<0) with NaN
df["Wind_Speed_Mph"] = df["Wind_Speed_Mph"].apply(
    lambda x: np.nan if x < 0 else x
)

# Print a message to show that data cleaning is done
print("Cleaned the invalid values\n")

# Step 4: Display selected columns to verify cleaning
df[["City", "State", "Country", "Humidity_Pct", "Wind_Speed_Mph"]]
```

Cleaned the invalid values

```
Out[99]:
```

	City	State	Country	Humidity_Pct	Wind_Speed_Mph
0	Columbus	OH	US	72	10.36
1	New York	NY	US	93	13.80
2	Chicago	IL	US	77	19.57
3	Los Angeles	CA	US	54	10.36
4	Houston	TX	US	84	10.36
5	Phoenix	AZ	US	27	1.01
6	Philadelphia	PA	US	97	8.05
7	San Antonio	TX	US	49	9.22
8	San Diego	CA	US	62	10.36
9	Dallas	TX	US	41	4.61

Step 7: Deriving Additional Weather Metrics

In this section, we create new columns in our weather dataset to provide more insights:

- **Temperature Range (Fahrenheit):** Calculates the difference between the maximum and minimum temperatures for each entry. This helps identify the daily temperature variation.
- **Daytime Indicator:** Determines whether a given timestamp falls during daytime. It checks if the timestamp is between the recorded sunrise and sunset times, returning `True` if it is daytime and `False` otherwise.
- **Confirmation Message:** Prints a message to indicate that the derived columns have been successfully added to the dataset.
- **Displaying Key Columns:** Shows important columns including `City`, `State`, `Country`, and the newly created `Temp_Range_F` and `Is_Daytime` for review.

In [100...

```
# Calculate temperature range in Fahrenheit (max - min)
df["Temp_Range_F"] = df["Temp_Max_F"] - df["Temp_Min_F"]

# Create a column to check if the timestamp is during the daytime
# True if current time is between sunrise and sunset, else False
df["Is_Daytime"] = df.apply(
    lambda row: row["Sunrise"] <= row["Timestamp"] <= row["Sunset"],
    axis=1
)

# Print a message to indicate that new columns were added
print("Added the derived columns\n")

# Display relevant columns, including the newly created ones
df[["City", "State", "Country", "Temp_Range_F", "Is_Daytime"]]
```

Added the derived columns

Out[100...

	City	State	Country	Temp_Range_F	Is_Daytime
0	Columbus	OH	US	3.31	False
1	New York	NY	US	3.29	False
2	Chicago	IL	US	2.92	False
3	Los Angeles	CA	US	4.86	True
4	Houston	TX	US	3.91	False
5	Phoenix	AZ	US	3.66	False
6	Philadelphia	PA	US	4.72	False
7	San Antonio	TX	US	5.37	False
8	San Diego	CA	US	3.84	True
9	Dallas	TX	US	2.63	False

Step 8: Dropping Unnecessary Columns from the Dataset

In this section, we clean our dataset by removing unnecessary columns.

- **Columns to Remove:** We identify metadata columns such as `Base`, `Visibility`, `Timezone`, and `Cod` that do not contribute to our analysis.
- **Conditional Dropping:** Only the columns that actually exist in the dataframe are removed, avoiding errors if some columns are missing.
- **Confirmation Message:** A message is printed to indicate that the unnecessary metadata has been successfully removed.
- **Updated Dataframe:** The dataframe is displayed after the columns are dropped, showing only the relevant data for analysis.

```
In [101... # List of columns we want to remove from the dataset
cols_to_drop = ["Base", "Visibility", "Timezone", "Cod"]

# Drop only the columns that exist in the dataframe
df = df.drop(columns=[c for c in cols_to_drop if c in df.columns])

# Print confirmation that unnecessary metadata columns were removed
print("Dropped the unnecessary metadata\n")

# Display the updated dataframe
df
```

Dropped the unnecessary metadata

Out[101...

	Dt	Id	Name	City	State	Country	Coord.Lon	Coord.Lat	1
0	1771637759	4509177	Columbus	Columbus	OH	US	-82.9988	39.9612	
1	1771637783	5128581	New York	New York	NY	US	-74.0060	40.7143	
2	1771637770	4887398	Chicago	Chicago	IL	US	-87.6500	41.8500	
3	1771637858	5368361	Los Angeles	Los Angeles	CA	US	-118.2437	34.0522	
4	1771637719	4699066	Houston	Houston	TX	US	-95.3633	29.7633	
5	1771637953	5308655	Phoenix	Phoenix	AZ	US	-112.0740	33.4484	
6	1771637777	4560349	Philadelphia	Philadelphia	PA	US	-75.1638	39.9523	
7	1771637701	4726206	San Antonio	San Antonio	TX	US	-98.4936	29.4241	
8	1771637754	5391811	San Diego	San Diego	CA	US	-117.1573	32.7153	
9	1771637817	4684904	Dallas	Dallas	TX	US	-96.7836	32.7668	

Step 9: Display Full Dataset with All Columns

In this section, we configure pandas to display the entire DataFrame without truncating columns or limiting column width.

- **display.max_columns** : Ensures all columns in the DataFrame are displayed, preventing truncation.
- **display.width** : Adjusts the output so that the DataFrame can expand to the full width of the notebook display.
- **Print Statement**: Adds a header or title for clarity before showing the DataFrame.
- **Displaying the DataFrame**: Shows the entire cleaned dataset in the notebook output.

```
In [102... # Set pandas option to display all columns without truncation
pd.set_option('display.max_columns', None)

# Set pandas option to display the full width of the DataFrame in output
pd.set_option('display.width', None)

# Print a title/header for clarity
print("Final Cleaned Dataset (All Columns):\n")

# Display the DataFrame
df
```

Final Cleaned Dataset (All Columns):

Out[102...

	Dt	Id	Name	City	State	Country	Coord.Lon	Coord.Lat	T
0	1771637759	4509177	Columbus	Columbus	OH	US	-82.9988	39.9612	
1	1771637783	5128581	New York	New York	NY	US	-74.0060	40.7143	
2	1771637770	4887398	Chicago	Chicago	IL	US	-87.6500	41.8500	
3	1771637858	5368361	Los Angeles	Los Angeles	CA	US	-118.2437	34.0522	
4	1771637719	4699066	Houston	Houston	TX	US	-95.3633	29.7633	
5	1771637953	5308655	Phoenix	Phoenix	AZ	US	-112.0740	33.4484	
6	1771637777	4560349	Philadelphia	Philadelphia	PA	US	-75.1638	39.9523	
7	1771637701	4726206	San Antonio	San Antonio	TX	US	-98.4936	29.4241	
8	1771637754	5391811	San Diego	San Diego	CA	US	-117.1573	32.7153	
9	1771637817	4684904	Dallas	Dallas	TX	US	-96.7836	32.7668	

Step 10: Transposing and Cleaning a DataFrame

In this section, we modify the structure of our DataFrame for easier analysis and better readability.

- **Transpose the DataFrame:** Switches rows and columns, which can help when we want to view data differently or align it for comparison.
- **Set Column Names:** Uses values from the "City" column to rename the columns, making the DataFrame more descriptive.
- **Remove Index Name:** Clears the index name for a cleaner display in tables.
- **Display the Transposed DataFrame:** Shows the updated DataFrame after transposing and renaming columns.

```
In [103... # Transpose the dataframe (swap rows and columns)
df_transposed = df.T

# Set the column names using the values from the "City" column
df_transposed.columns = df["City"].tolist()

# Remove the index name for cleaner display
df_transposed.index.name = None

# Display the transposed dataframe
df_transposed
```

Out[103...

	Columbus	New York	Chicago	Los Angeles	Houston	Ph
Dt	1771637759	1771637783	1771637770	1771637858	1771637719	17716
Id	4509177	5128581	4887398	5368361	4699066	53
Name	Columbus	New York	Chicago	Los Angeles	Houston	Ph
City	Columbus	New York	Chicago	Los Angeles	Houston	Ph
State	OH	NY	IL	CA	TX	
Country	US	US	US	US	US	
Coord.Lon	-82.9988	-74.006	-87.65	-118.2437	-95.3633	-11
Coord.Lat	39.9612	40.7143	41.85	34.0522	29.7633	33
Temperature_F	37.9	36.48	28.62	54.77	74.68	
Feels_Like_F	30.88	27.73	16.02	52.48	75.79	
Temp_Min_F	36.09	34.29	27.21	52.47	72.1	
Temp_Max_F	39.4	37.58	30.13	57.33	76.01	
Pressure	1013	1005	1014	1017	1008	
Humidity_Pct	72	93	77	54	84	
Sea_Level	1013	1005	1014	1017	1008	
Grnd_Level	983	1004	991	997	1006	
Wind_Speed_Mph	10.36	13.8	19.57	10.36	10.36	
Wind_Direction_Deg	270	360	270	270	160	
Gust	19.57	NaN	34.52	NaN	NaN	
Clouds.All	20	100	100	0	75	
Sys.Type	2	1	2	2	2	
Sys.Id	2079804	4610	2005153	2009067	2001415	20
Country_Code	US	US	US	US	US	
Sys.Sunrise	1771589906	1771587807	1771591167	1771597947	1771592196	17715
Sys.Sunset	1771629200	1771626983	1771630171	1771638076	1771632845	17716
Weather_Id	801	300	804	800	803	
Weather_Main	Clouds	Drizzle	Clouds	Clear	Clouds	C
Weather_Description	Few Clouds	Light Intensity Drizzle	Overcast Clouds	Clear Sky	Broken Clouds	Sca C

	Columbus	New York	Chicago	Los Angeles	Houston	Ph
Weather_Icon	02n	09n	04n	01d	04n	
Timestamp	2026-02-21 01:35:59	2026-02-21 01:36:23	2026-02-21 01:36:10	2026-02-21 01:37:38	2026-02-21 01:35:19	2026-01-01
Sunrise	2026-02-20 12:18:26	2026-02-20 11:43:27	2026-02-20 12:39:27	2026-02-20 14:32:27	2026-02-20 12:56:36	2026-01-14
Sunset	2026-02-20 23:13:20	2026-02-20 22:36:23	2026-02-20 23:29:31	2026-02-21 01:41:16	2026-02-21 00:14:05	2026-01-01
Temp_Range_F	3.31	3.29	2.92	4.86	3.91	
Is_Daytime	False	False	False	True	False	

Ethical Implications of Data Wrangling

Data Sourcing and Credibility

The weather data was sourced directly from the OpenWeatherMap API, a reputable and publicly documented service. This ensures the data is reliable and credible. The API is publicly accessible, and usage complies with OpenWeatherMap's terms of service, which covers legal and regulatory guidelines for this type of public weather data. The dataset was obtained ethically through authorized API access, without collecting any personal or sensitive information.

Data Transformations and Assumptions

During data wrangling, several transformations were applied to improve readability and usability. These included renaming columns for clarity, flattening nested JSON structures, standardizing text casing, converting temperatures from Kelvin to Fahrenheit, and formatting timestamps into human-readable date-time formats. Additionally, invalid values, such as humidity outside the 0–100% range, were removed, and derived metrics were created. While these steps enhance usability, assumptions were made, for example, treating out-of-range values as invalid, which could inadvertently remove rare but valid edge-case data.

Risks of Transformations

While these transformations improve usability, they also introduce risks. Incorrect assumptions or rounding could inadvertently remove valid data or reduce precision, potentially affecting decisions based on this dataset, such as travel planning, safety measures, or climate analysis. Transforming nested JSON and deriving new metrics could also introduce errors if not carefully verified.

Ethical Mitigation Measures

To address potential ethical concerns, all transformation steps were clearly documented, and the original raw data fields were preserved before cleaning. Each assumption and its justification were transparently stated. Cross-verification of API data and careful handling of outliers were used to minimize the risk of errors. These measures ensure that the final dataset is both usable and ethically responsible, while maintaining the credibility and integrity of the information.